Indian Institute of Technology Palakkad

CS5657: Geometric CV

Mini-Project 1: Estimating Camera Calibration

Submitted By:

Vuppula Hemanth **142201022**
P. Chakradhar Reddy **112201022**

September 2, 2025

Contents

1 Introduction

Camera calibration is a fundamental task in computer vision and graphics, essential for applications ranging from 3D reconstruction to augmented reality. The goal is to determine the camera's intrinsic and extrinsic parameters, which collectively form the **perspective projection matrix (M)**. This 3x4 matrix mathematically describes how 3D points in the real world are projected onto the 2D image plane of the camera.

This project focuses on computing this matrix using a single RGB image and its corresponding depth map, sourced from the NYU Depth V2 dataset. For our experiment, we selected a representative indoor scene—a bedroom—to serve as our test case. By leveraging the depth information and the camera's known intrinsic properties, we can establish correspondences between 2D pixels and 3D world points. These correspondences are then used in the Direct Linear Transformation (DLT) algorithm to robustly estimate the projection matrix.

2 Methodology

The process involves several key steps, from data preparation to matrix validation.

2.1 From Pixels to World Coordinates

Given an RGB image and a depth map, we can calculate the real-world 3D coordinates (X, Y, Z) for any pixel (u, v) if the camera's intrinsic parameters are known. The relationship is defined by the inverse pinhole camera model:

$$X = \frac{(v - c_x) \cdot Z_d}{f_x} \tag{1}$$

$$Y = \frac{(u - c_y) \cdot Z_d}{f_y} \tag{2}$$

$$Z = Z_d \tag{3}$$

where (u, v) are the pixel coordinates (row, column), (c_x, c_y) is the principal point, (f_x, f_y) are the focal lengths, and Z_d is the depth value at that pixel. This step allows us to build a set of 2D-3D point correspondences.

2.2 The Direct Linear Transformation (DLT) Algorithm

The DLT algorithm provides a linear method for finding the projection matrix M. A 3D point $P_w = [X, Y, Z, 1]^T$ is projected to a 2D image point $p_i = [u, v, 1]^T$ by the equation $p_i \cong MP_w$. By expanding this, we derive two linear equations for each point correspondence:

$$u = \frac{m_1 \cdot P_w}{m_3 \cdot P_w} \implies u(m_3 \cdot P_w) - (m_1 \cdot P_w) = 0 \tag{4}$$

$$v = \frac{m_2 \cdot P_w}{m_3 \cdot P_w} \implies v(m_3 \cdot P_w) - (m_2 \cdot P_w) = 0 \tag{5}$$

where m_1, m_2, m_3 are the rows of the matrix M.

For n correspondences, we can construct a $2n \times 12$ matrix A such that $Ap = 0$, where p is a 12-element vector containing the flattened entries of M.

2.3 Solving for the Projection Matrix

The system $Ap = 0$ is a homogeneous system of linear equations. The optimal solution for p (in a least-squares sense) is found by performing Singular Value Decomposition (SVD) on matrix

A. The solution is the singular vector corresponding to the smallest singular value, which is the last row of the V^T matrix. This 1×12 vector is then reshaped to form the 3x4 projection matrix M.

2.4 Validation via Reprojection Error

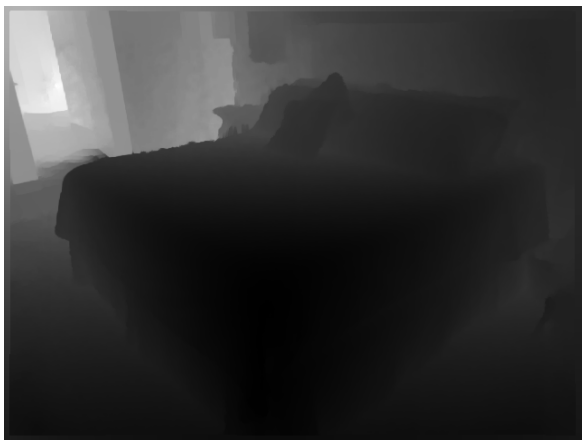
To validate the accuracy of our estimated matrix M, we reproject the original 3D world points back onto the 2D image plane. The mean reprojection error is the average Euclidean distance between the original pixel coordinates (u, v) and the reprojected coordinates (u', v'). A small error indicates a successful calibration.

3 Experimental Setup & Results

3.1 Selected Scene and Camera Parameters

For our analysis, we chose a bedroom scene from the NYU Depth V2 dataset. This scene provides a rich set of features at varying depths, making it a good candidate for calibration. Figure 1 shows the RGB image and its corresponding depth map.

(a) Original RGB Image



(b) Corresponding Depth Map (Grayscale)

Figure 1: The bedroom scene used for camera calibration.

The provided intrinsic camera parameters are listed in Table 1.

Table 1: Intrinsic Camera Parameters

Parameter	Value
f_x	518.85790117450188
f_y	519.46961112127485
c_x	325.58244941119034
c_y	253.73616633400465

3.2 Estimated Projection Matrix (M)

Using $n = 10$ randomly sampled correspondence points from the available 306,661 valid pixels (where depth > 0), the DLT algorithm yielded the following estimated 3x4 projection matrix:

$$M_{est} = \begin{bmatrix} -1.857 \times 10^{-13} & -6.167 \times 10^{-1} & -3.012 \times 10^{-1} & 8.393 \times 10^{-13} \\ -6.160 \times 10^{-1} & -1.360 \times 10^{-13} & -3.865 \times 10^{-1} & 1.767 \times 10^{-12} \\ -1.943 \times 10^{-16} & -4.163 \times 10^{-17} & -1.187 \times 10^{-3} & -1.350 \times 10^{-15} \end{bmatrix}$$

3.3 Validation and Visualization

The quality of the estimated matrix M was evaluated by calculating the mean reprojection error. For our $n = 10$ sample points, the error was found to be 3.354×10^{-11} pixels. This near-zero error demonstrates the high fidelity of the estimated matrix and the correctness of our implementation.

Figure 2 provides a visual confirmation of this accuracy. The original points (red circles) are overlaid with their reprojected counterparts (lime crosses), showing a near-perfect match.

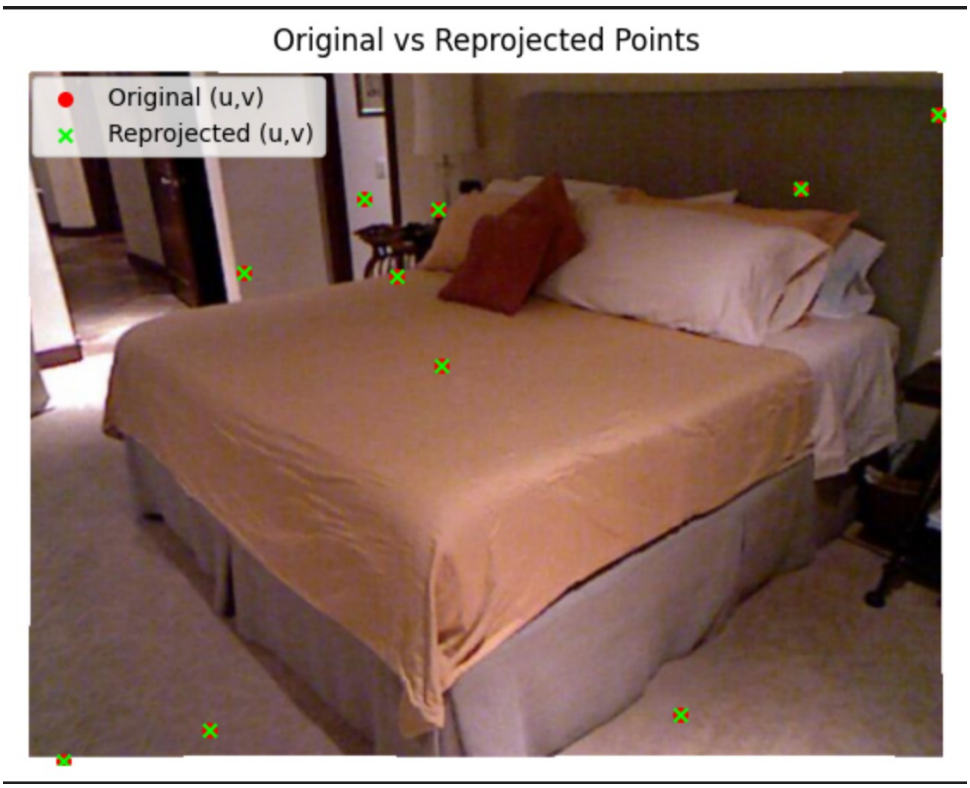


Figure 2: Visualization of original sampled points vs. reprojected points using the estimated matrix M. The perfect overlap validates the accuracy of the computed matrix.

4 Conclusion and Future Work

This project successfully demonstrated the process of camera calibration from a single view with depth information. By correctly implementing the Direct Linear Transformation algorithm, we were able to compute the 3x4 projection matrix with a high degree of accuracy, as validated by a mean reprojection error of approximately 3.35×10^{-11} pixels. The results confirm a solid understanding of the principles of perspective projection and camera geometry.

For future work, several improvements could be implemented:

- **Data Normalization:** To improve numerical stability, normalizing the 2D and 3D point coordinates before running DLT is a standard practice.
- **Non-linear Optimization:** The DLT solution could be used as an initial guess for a non-linear optimization algorithm (like Levenberg-Marquardt) to further refine the matrix by minimizing the true geometric error.
- **Robust Estimation (RANSAC):** If the correspondence points contained outliers, using a robust method like RANSAC would allow the algorithm to find the best matrix based on a consensus set of inlier points.

A Full Python Code

```python
import cv2
import numpy as np
import matplotlib.pyplot as plt

# --- 1. Load Data ---
img = cv2.imread('img.png')
depth = cv2.imread('dep.png')

# Convert depth to a single-channel grayscale array
dep = depth[:, :, 0]

# --- 2. Camera and DLT Setup ---
# Provided camera intrinsic parameters
fx = 5.1885790117450188e+02
fy = 5.1946961112127485e+02
cx = 3.2558244941119034e+02
cy = 2.5373616633400465e+02

def pixel_to_world_xyz(u: int, v: int):
    """Convert pixel (row=u, col=v) to real-world (X,Y,Z)."""
    Z = float(dep[u, v])
    X = (v - cx) * Z / fx
    Y = (u - cy) * Z / fy
    return (X, Y, Z)

def build_dlt_matrix(points):
    """Construct the A matrix for DLT from a list of (u,v) pixels."""
    if len(points) < 6:
        raise ValueError("DLT requires at least 6 correspondences.")
    A = []
    for (u, v) in points: # u is row (y), v is col (x)
        X, Y, Z = pixel_to_world_xyz(u, v)
        Q = [X, Y, Z, 1.0]
        # Equation for the x-coordinate (v)
        A.append(Q + [0,0,0,0] + [-v*X, -v*Y, -v*Z, -v])
        # Equation for the y-coordinate (u)
        A.append([0,0,0,0] + Q + [-u*X, -u*Y, -u*Z, -u])
    return np.array(A, dtype=float)

def estimate_projection_matrix(points):
    """Estimate 3x4 projection matrix P via DLT given (u,v) pixel list."""
    A = build_dlt_matrix(points)
    _, _, Vt = np.linalg.svd(A)
    P = Vt[-1].reshape(3, 4)
    return P

def mean_reprojection_error(P, pts_uv, pts_XYZ1):
    """Calculate the mean reprojection error."""
    proj = P @ pts_XYZ1.T
    proj = proj / proj[-1] # Normalize homogeneous coordinates
    u_proj, v_proj = proj[0], proj[1]
    u_orig, v_orig = pts_uv[:,0], pts_uv[:,1]
    errors = np.sqrt((u_proj - u_orig)**2 + (v_proj - v_orig)**2)
    return float(np.mean(errors))

# --- 3. Main Execution and Visualization ---
# Find pixels with valid depth information
valid_pixels = np.where(dep > 0)
if valid_pixels[0].size < 6:
    raise ValueError("Not enough valid depth pixels to sample from.")

```

```python
# Sample 10 random valid points for calibration
requested_points = 10
num_points = min(requested_points, valid_pixels[0].size)
print(f"Sampling {num_points} correspondences (requested {requested_points}, available {valid_pixels[0].size}).")

rand_indices = np.random.choice(valid_pixels[0].size, num_points, replace=False)
sample_points_uv = [(int(valid_pixels[0][i]), int(valid_pixels[1][i])) for i in rand_indices]

# Estimate the projection matrix
P_estimated = estimate_projection_matrix(sample_points_uv)

# Prepare arrays for reprojection error calculation
pts_uv_array = np.array(sample_points_uv, dtype=int)
pts_XYZ_array = np.array([pixel_to_world_xyz(u, v) for (u, v) in sample_points_uv], dtype=float)
pts_XYZ1_array = np.hstack([pts_XYZ_array, np.ones((pts_XYZ_array.shape[0], 1))])

# Calculate and print results
mean_err = mean_reprojection_error(P_estimated, pts_uv_array, pts_XYZ1_array)
print("Mean reprojection error:", mean_err)
print("\nEstimated P:\n", P_estimated)

# Visualize original vs. reprojected points
projected_pts_homogeneous = P_estimated @ pts_XYZ1_array.T
projected_pts = projected_pts_homogeneous / projected_pts_homogeneous[-1]
u_projected, v_projected = projected_pts[0], projected_pts[1]

plt.figure(figsize=(8, 7))
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.scatter(pts_uv_array[:,1], pts_uv_array[:,0], c='red', marker='o', s=40, label='Original (u,v)')
plt.scatter(v_projected, u_projected, c='lime', marker='x', s=40, label='Reprojected (u,v)')
plt.title('Original vs. Reprojected Points')
plt.legend()
plt.axis('off')
plt.show()
```

Listing 1: Full Python script for camera calibration.